

Version 2.0

January 2026



OPERATING SYSTEMS PRACTICUM

MODULE 3 - USER, GROUP, AND FILE MANAGEMENT

Author:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
THEORETICAL BACKGROUND.....	4
1. Basic Concepts of Users and Groups.....	4
1.1 Understanding Users.....	4
1.2 Understanding Groups.....	4
1.3 User and Group Relationship.....	4
2. User and Group Management.....	5
2.1 Basic User Management Commands.....	5
EXERCISE 1 : The Administrator (Management Commands).....	10
2.2 Basic Group Management Commands.....	10
2.3 User and Group Configuration Files.....	12
EXERCISE 2 : The Identity Check (Users & Groups).....	12
EXERCISE 3 : The Auditor (System Files).....	13
2.4 Understanding Sudo and the Superuser (Root).....	13
3. Basic Files and Permissions.....	14
3.1 File Permission Concepts.....	14
3.2 Basic File Permission Commands.....	15
3.3 File Linking (Hard Link vs Soft Link).....	18
3.4 File Ownership and its Relation to Users & Groups.....	19
EXERCISE 4 : The Gatekeeper (File Permissions).....	20
EXERCISE 5: The Guardian.....	20
PRACTICAL APPLICATIONS.....	22
PRACTICE 1.....	22
PRACTICE 2.....	22
DEMO - CASE STUDY.....	22
Demo Rules.....	24
ASSESSMENT RUBRIC.....	25
ASSESSMENT SCALE.....	25



INTRODUCTION

OBJECTIVES

1. Understand Linux user and group concepts
2. Manage users, groups, and file ownership
3. Apply basic file permissions for system security

MODULE TARGETS

1. Identify and differentiate users and groups
2. Create, modify, and delete users and groups
3. Assign ownership and set file permissions
4. Verify user, group, and permission configuration

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop or personal computer
3. Rocky Linux
4. Virtual Box
5. Strong motivation and commitment to learning

KEYWORDS

User, Group, UID, GID, File Ownership, Permission, Root, Superuser, Security, Access Control



THEORETICAL BACKGROUND

1. Basic Concepts of Users and Groups

In Linux-based operating systems, the concepts of **users** and **groups** form the foundation for managing access, security, and system resources. A solid understanding of users and groups makes it easier to manage files, directories, and permissions effectively.

1.1 Understanding Users

A user is an entity or account that has permission to access the system. Each user can log in, run applications, and manage files according to the permissions assigned to them. Not all users have the same level of access. There are regular users and the root user (also known as the superuser).

- **The Root User:** Has full privileges to perform any operation on the system, including changing system configurations, deleting critical files, or adding new users.
- **Regular Users:** Have limited permissions to prevent accidental damage or unauthorized system changes.
- **User ID (UID):** Each user is identified by a numeric UID, which distinguishes one user from another. The UID is crucial when the system determines file ownership and access rights.

1.2 Understanding Groups

A group is a collection of users. The main purpose of groups is to simplify permission management. Instead of setting permissions for each user individually, they can be placed into a group, and permissions can be granted to the group as a whole.

- **Default Groups:** Automatically created when a user account is created. The user usually belongs to this group by default (often called a **Primary Group**).
- **Custom Groups:** Created based on specific needs, such as a "development" team needing access to a specific project folder.
- **Group ID (GID):** Every group is identified by a numeric GID, which identifies the user's primary group membership.

1.3 User and Group Relationship

Every user has a **primary group**, which is the default group associated with the user when creating files or folders. Additionally, a user can belong to multiple **secondary groups** that provide additional access rights. This relationship is crucial because file permissions in Linux depend heavily on user and group associations. To see how the system implements these concepts, we use the **id command**. This utility displays the real and effective User ID (UID) and Group ID (GID).



2. User and Group Management

2.1 Basic User Management Commands

Managing users in Linux involves a set of commands that allow you to create, modify, and delete user accounts. These commands are essential for system administration and maintaining proper access control.

A. ID

This command shows the UID (User ID), GID (Primary Group ID), and all group memberships of a user. It is useful to quickly verify a user's identity and group associations.

```
praktikumso@localhost:~/LatihanSO1$ id
uid=1000(praktikumso) gid=1000(praktikumso) groups=1000(praktikumso),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Explanation of the output :

- The **id** Command: This command serves as a verification tool to check the current user's identity and privileges before performing system tasks.
- **uid=1000(praktikumso)**: The User ID (UID) is 1000. In most Linux distributions, UID 0 is reserved for the root user, while UIDs starting from 1000 are assigned to regular users. This confirms that "praktikumso" is a regular account with limited permissions.
- **gid=1000(praktikumso)**: The Group ID (GID) identifies the primary group of the user. Any files created by this user will, by default, be associated with this group ID.
- **groups=...10(wheel)**: This lists all groups the user belongs to. The inclusion of the **wheel** group (ID 10) is significant because it grants this regular user the authority to execute administrative commands via **sudo**.

B. whoami

While the **id** command provides a comprehensive look at UIDs, GIDs, and group memberships, there are times when you only need a simple confirmation of your current account name. For this purpose, Linux provides the **whoami** command.

```
praktikumso@localhost:~/LatihanSO1$ whoami
praktikumso
```

In the example above, the output **praktikumso** confirms that the user is logged in as the regular user "praktikumso," which aligns with the UID 1000 identified previously using the **id** command.



C. useradd

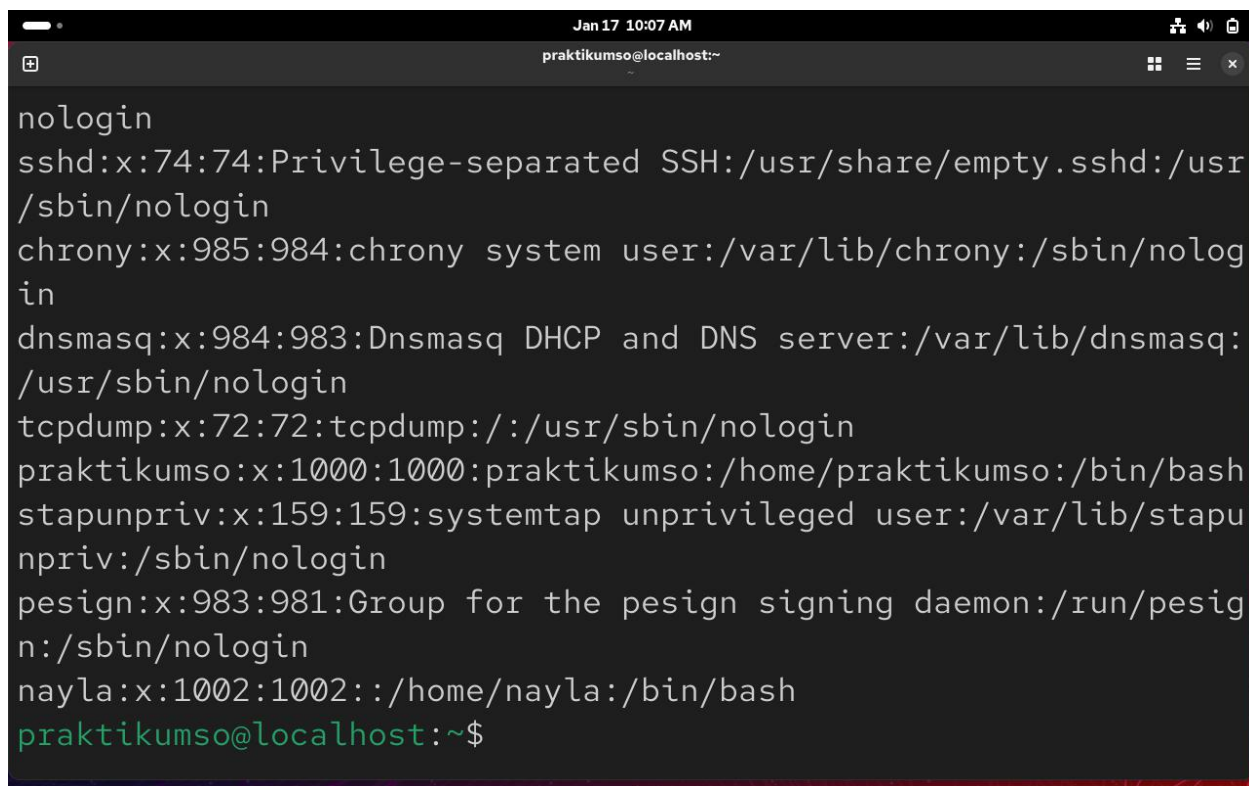
The `useradd` command is used to create a new user account on the system.

```
praktikumso@localhost:~$ sudo useradd -m nayla
```

-m Flag: This is a critical option that tells the system to create a **Home Directory** (e.g., `/home/new_username`) for the new user. Without this, the user will not have a private space to store files.

Most user management commands in Linux require `sudo` because they affect the entire system. Managing users is an administrative task related to security, so only the root user or users with elevated privileges are allowed to perform it. User data is stored in protected system files such as `/etc/passwd` and `/etc/shadow`, which regular users cannot modify. Requiring `sudo` helps prevent unauthorized or accidental changes and ensures system integrity.

In Linux, all user account information is stored in a specific configuration file located at `/etc/passwd`. You can verify if a user has been successfully created by viewing this file.



```
Jan 17 10:07 AM
praktikumso@localhost:~
nologin
sshd:x:74:74:Privilege-separated SSH:/usr/share/empty.sshd:/usr
/sbin/nologin
chrony:x:985:984:chrony system user:/var/lib/chrony:/sbin/nolog
in
dnsmasq:x:984:983:Dnsmasq DHCP and DNS server:/var/lib/dnsmasq:
/usr/sbin/nologin
tcpdump:x:72:72:tcpdump:/:/usr/sbin/nologin
praktikumso:x:1000:1000:praktikumso:/home/praktikumso:/bin/bash
stapunpriv:x:159:159:systemtap unprivileged user:/var/lib/stapu
npriv:/sbin/nologin
pesign:x:983:981:Group for the pesign signing daemon:/run/pesig
n:/sbin/nologin
nayla:x:1002:1002:./home/nayla:/bin/bash
praktikumso@localhost:~$
```

As shown in the image, after running the command, you can see the new entry at the bottom:
`nayla:x:1002:1002:./home/nayla:/bin/bash.`



How to read the output:

- **nayla**: The username you just created.
- **x**: Indicates that the password is encrypted and stored in `/etc/shadow`.
- **1002**: The **User ID (UID)**. Notice it is higher than `praktikumso` (1000), indicating it is a new regular user.
- **1002**: The **Group ID (GID)** for the user's primary group.
- **/home/nayla**: The path to the home directory created by the `-m` flag.
- **/bin/bash**: The default shell for this user.

D. passwd

Creating a user account is only the first step. For a user to be able to log in and access the system, they must have a secure password. The `passwd` command is used to set or change the password of a user account. This ensures that only the authorized individual can access the specific user profile.

```
praktikumso@localhost:~$ sudo passwd nayla
New password:
Retype new password:
passwd: password updated successfully
```

To verify that a password has been successfully created and secured, you can inspect the `/etc/shadow` file using administrative privileges. Since this file contains sensitive authentication data, you must use the command `sudo tail /etc/shadow` to view the most recent entries.

```
praktikumso@localhost:~$ sudo tail /etc/shadow
gdm:!:20462::::::
gnome-initial-setup:!:20462::::::
sshd:!:20462::::::
chrony:!:20462::::::
dnsmasq:!:20462::::::
tcpdump:!:20462::::::
praktikumso:$y$j9T$zm9XSTVMO3qWF1XPZ0HjY5.t$3YMWGvIYDq2zUHPKz2q
6h5FSTrQvQmMJh0rtTFfW6w0::0:99999:7:::
stapunpriv:!*:20462::::::
pesign:!:20462::::::
nayla:$y$j9T$rvrUYx7jnybhmmn0pSu7s/$TxGb5AupWhskfX/5LxuAsjIBL8C
tD/DYR2eyGRUx.aA:20470:0:99999:7:::
```



For the user "nayla," you can see a long string of random characters starting with symbols like `$y$j9T$...`. This is the **password hash**, confirming that the password is not stored in plain text but is encrypted for security.

Beyond setting a password, Linux also allows administrators to manage its "age" or validity period to enhance system security. This is done using the **chage** (Change Age) command. By setting a password policy, you can force users to change their passwords periodically, ensuring that an account remains secure even if a password was previously compromised. All metadata related to password aging—such as the last change date, the maximum number of days before a change is required, and when the account expires—is stored alongside the encrypted hash in the `/etc/shadow` file. To view the current aging status of a user:

```
praktikumso@localhost:~/dev_project$ sudo chage -l raven
[sudo] password for praktikumso:
Last password change                : Feb 17, 2026
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

- To set how many days in advance a user will receive a warning before their password expires: `praktikumso@localhost:~$ sudo chage -W [days] [username]`
- To set a specific maximum number of days a password remains valid before the user is required to change it: `praktikumso@localhost:~$ sudo chage -M [days] [username]`

```
praktikumso@localhost:~/dev_project$ sudo chage -M 90 -W 7 raven
praktikumso@localhost:~/dev_project$ sudo chage -l raven
Last password change                : Feb 17, 2026
Password expires                    : May 18, 2026
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
praktikumso@localhost:~/dev_project$
```

The image above demonstrates the process of modifying the password aging policy for the user **raven**. By using the command `sudo chage -M 90 -W 7 raven`, the administrator sets the maximum password validity to 90 days (`-M`) and configures a warning period of 7 days (`-W`) before the password expires.



E. usermod

The **usermod** command is a powerful administrative tool used to modify an existing user's attributes, such as their home directory or group affiliations.

Before performing modifications, it is essential to follow a "Check, Execute, and Verify" workflow to ensure the system configuration is correct.

1. First, check which groups the user "nayla" currently belongs to.

```
praktikumso@localhost:~$ groups nayla
nayla : nayla
```

This shows that the user currently only belongs to her primary group.

2. Next, create a custom group named "praktikan" that will be used for collective permission management.

```
praktikumso@localhost:~$ sudo groupadd praktikan
```

3. Now, add the user "nayla" to the newly created "praktikan" group.

```
praktikumso@localhost:~$ sudo usermod -aG praktikan nayla
```

The -aG Flag: Using **-a** (append) is critical; if you omit it and only use **-G**, the user will be removed from all other secondary groups and only belong to "praktikan".

4. Check the user's attributes again to confirm the relationship was established successfully.

```
praktikumso@localhost:~$ id nayla
uid=1002(nayla) gid=1002(nayla) groups=1002(nayla),1003(praktikan)
```

The user "nayla" now has two group associations. The user remains in its primary group (1002) but is now also a member of the secondary group **praktikan** (1003). The user will now inherit any file permissions granted to the "praktikan" group.

F. userdel

The final stage of user management is the removal of accounts that are no longer needed using the **userdel** command. To completely remove a user and all their personal files, you should use the command **sudo userdel -r username**.

```
praktikumso@localhost:~$ sudo userdel -r nayla
praktikumso@localhost:~$ id nayla
id: 'nayla': no such user
```

The **-r** Flag (Recursive) is a powerful option that deletes not only the user account from the system files (like **/etc/passwd**) but also removes the user's **Home Directory** and their mail spool. Without this flag, the user's files will remain on the disk as "orphaned" files with no associated owner.



G. Switching Users with su

In Linux, you often need to switch between different accounts to perform specific tasks or verify if file permissions are working correctly for different users. The `su` command (Substitute User) is the primary tool for this. To switch to another user, use the command:

```
praktikumso@localhost:~$ su [username]
```

```
praktikumso@localhost:~$ su - raven
Password:
Last login: Tue Feb 17 22:44:18 WIB 2026 on pts/0
raven@localhost:~$ exit
logout
praktikumso@localhost:~$
```

It is highly recommended to use `su - [username]`. The hyphen (-) ensures that the system loads the target user's full environment, including their specific home directory and shell settings. Additional information, when switching to a regular user, you must enter that specific user's password. To return to your original account, simply type `exit`.

EXERCISE 1 : The Administrator (Management Commands)

1. Create a new group named `dev_team`.
2. Create a new user named `developer1`.
3. Assign `developer1` to the `dev_team` group.
4. Set a temporary password for the new user `developer1`.
5. Add the user `developer1` to the `wheel` group so they can perform administrative tasks.
6. Switch to the `developer1` account using the `su -` command and verify the identity using `whoami`.
7. While logged in as `developer1`, try to view the `/etc/shadow` file using `sudo`.
8. Return to the `praktikumso` account by using the `exit` command.

2.2 Basic Group Management Commands

Groups serve as a fundamental tool in Linux to organize users and simplify permission management. Instead of assigning permissions to individuals, administrators can grant rights to a group, and all members will automatically inherit those privileges.

A. groupadd

The `groupadd` command is used to create a new custom group on the system. This is typically the first step when setting up a collaborative environment, such as creating a shared folder for a specific team.



```
praktikumso@localhost:~$ sudo groupadd praktikan
```

You can verify that a group has been successfully created by searching for its name in the system's group database using the command `grep "groupname" /etc/group`.

```
praktikumso@localhost:~$ grep "praktikan" /etc/group
praktikan:x:1003:
```

B. groupmod

The `groupmod` command is used to modify the attributes of an existing group. While groups are primarily identified by their GID by the system, administrators often need to change the human-readable group name to reflect organizational changes. To rename a group, the syntax is `sudo groupmod -n newgroupname oldgroupname`.

```
praktikumso@localhost:~$ sudo groupmod -n praktikan2026 praktikan
```

The **-n Flag (New Name)** specifies the new name for the group. It is important to note that while the name changes, the **GID remains the same**. This ensures that file permissions associated with that GID are not broken.

```
praktikumso@localhost:~$ grep "praktikan2026" /etc/group
praktikan2026:x:1003:
```

You should see the new name followed by the same GID that was previously assigned to the old group name.

C. groupdel

When a group is no longer needed, for example, if a project has ended or a department has been dissolved, you can remove it using the `groupdel` command.

```
praktikumso@localhost:~$ sudo groupdel praktikan2026
praktikumso@localhost:~$ grep "praktikan2026" /etc/group
praktikumso@localhost:~$
```

Important Considerations that you cannot delete a group if it is the **primary group** of an existing user. You must first delete the user or change their primary group using `usermod`.

D. groups

While the `id` command provides a deep dive into numeric IDs, the `groups` command is a more straightforward tool used to display only the names of the groups a specific user belongs to. It is the most common way to quickly verify if a user has been successfully added to a secondary group.

```
praktikumso@localhost:~$ groups nayla
nayla : nayla
```



The result `nayla : nayla` indicates that the user "nayla" currently only belongs to its **primary group** (which shares its name). This confirms that the user has not yet been added to any **secondary groups**, such as the "praktikan".

2.3 User and Group Configuration Files

Linux stores user and group information in several critical system files. Understanding the structure of these files is essential for troubleshooting access issues and manual system management.

A. /etc/passwd

This file is the primary database for all user accounts. Every line represents one user and contains seven fields separated by colons. It includes the username, a placeholder for the password (usually 'x'), the **UID**, the **Primary GID**, a comment field, the **Home Directory**, and the **Default Shell**. For security reasons, actual passwords are no longer stored here to prevent unauthorized access to encrypted credentials.

B. /etc/group

The `/etc/group` file defines the groups existing on the system. Each entry displays the **Group Name**, a password placeholder, the **GID**, and a list of all users who belong to that group as **secondary members**. This is the file that the system references whenever you use the `groups` command or modify memberships with `usermod -aG`.

C. /etc/shadow

The `/etc/shadow` file is the most sensitive of the three, as it stores the **encrypted password hashes** for all users. To maintain maximum security, this file is readable only by the **root user**. In addition to the encrypted password, it contains metadata such as the date of the last password change and the account's expiration information. If the second field for a user in this file contains a `!` or `*`, it indicates that the account is locked or has no password assigned.

EXERCISE 2 : The Identity Check (Users & Groups)

1. Use the command to display your current username.
2. Use the command to list all the groups your current user belongs to.
3. View the **configuration file** that stores user account information to verify your user exists in the system.
4. Use the `chage -l developer1` command to audit the password aging and expiration details for your user.
5. Modify the security policy by setting a maximum password validity period (e.g., `-M 90`) and a warning period (e.g., `-W 7`).
6. Run the `chage -l` command again to verify that your new security policies have been successfully applied.



EXERCISE 3 : The Auditor (System Files)

1. Use a command to view the last 5 lines of the `/etc/passwd` file to see your newly created user.
2. Use a command to view the `/etc/group` file to verify that the `dev_team` group exists and contains the correct members.
3. Use `sudo tail /etc/shadow` to view the encrypted password hash and aging metadata.

2.4 Understanding Sudo and the Superuser (Root)

In a Linux environment, not all users are created equal. To maintain system stability and security, Linux strictly separates administrative powers from everyday tasks. Understanding the boundary between a **Regular User** and the **Superuser (Root)** is fundamental for any system administrator.

A. The Superuser (Root)

The **root** user is the most powerful account on a Linux system, often referred to as the "superuser." This account has an ID (UID) of `0` and possesses unrestricted access to every file, command, and configuration setting. While this level of power is necessary for system maintenance, it is also dangerous; a single accidental command (like `rm -rf /`) can destroy the entire operating system instantly.

B. Regular Users

Regular users are accounts created for day-to-day work, typically assigned UIDs starting from `1000`. These users have restricted permissions, meaning they can only modify files within their own home directory and cannot access sensitive system configuration files like `/etc/shadow`.

C. The `sudo` Command (SuperUser DO)

Since logging in as "root" is considered a security risk, Linux uses a mechanism called **sudo**. The `sudo` command allows a permitted user to execute a command as the superuser. This follows the **Principle of Least Privilege**: you work as a regular user most of the time and only "elevate" your privileges when absolutely necessary.

D. Granting Sudo Privileges (The 'wheel' Group)

To give a new user administrative powers, you don't make them "root." Instead, you add them to a specific administrative group. In Rocky Linux, this group is called **wheel**.

- **The Workflow:** If a user belongs to the `wheel` group, they can use `sudo`.
- **Command Example:** Adding a user to the administrative group:




```
praktikumso@localhost:~$ sudo useradd -m raven
praktikumso@localhost:~$ id raven
uid=1002(raven) gid=1003(raven) groups=1003(raven)
praktikumso@localhost:~$ sudo usermod -aG wheel raven
praktikumso@localhost:~$ id raven
uid=1002(raven) gid=1003(raven) groups=1003(raven),10(wheel)
praktikumso@localhost:~$
```

The image above demonstrates how to grant administrative access to the user **raven** by using the command `sudo usermod -aG wheel raven`. As shown in the verification through the `id raven` command, the user has been successfully added to the **wheel** group, thereby inheriting `sudo` privileges.

E. Temporary Session Elevation (`sudo -i`)

Sometimes, an administrator needs to perform many tasks in a row. Instead of typing `sudo` before every command, you can start a temporary root shell.

```
praktikumso@localhost:~$ sudo -i
root@localhost:~#
```

The prompt will change from `$` to `#`, indicating you are now acting as root.

3. Basic Files and Permissions

After managing users and groups, it is essential to understand how Linux protects data through **File Permissions**. This system ensures that only authorized accounts can interact with specific files and directories.

3.1 File Permission Concepts

Each file and directory in Linux is assigned a set of access rights divided into three types and applied to three categories of users.

A. Permission Types

- **Read (r)**: Allows viewing file contents or listing files within a directory.
- **Write (w)**: Allows modifying file content or creating/deleting files within a directory.
- **Execute (x)**: Allows running a file as a program or entering a directory.

B. User Categories

Permissions are defined separately for:

1. **Owner**: The specific user who owns the file.
2. **Group**: All users who are members of the group assigned to the file.



3. **Others:** Every other user on the system who is not the owner or a group member.

C. Default Permissions (umask)

When a new file or directory is created, the system assigns default permissions based on the **umask** (user mask). This value "masks" or subtracts permissions from the maximum possible set to ensure new files are secure by default.

3.2 Basic File Permission Commands

Understanding file permissions in Linux requires not only knowing the commands, but also being able to read and interpret their output correctly. The following commands are the most commonly used tools for viewing and managing file permissions.

A. ls -l

The `ls -l` command displays detailed information about files and directories. This includes the file type, permission bits, number of links, owner, group, file size, and last modification time.

```
praktikumso@localhost:~/Documents$ ls -l Latihan1.txt
-rw-r--r--. 1 praktikumso praktikumso 0 Jan 17 11:28 Latihan1.txt
```

In the `ls -l` output, the first column displays a 10-character string such as `-rw-r--r--`, which acts as a technical summary of the file's type and its security settings. The first character indicates the file type, where a dash (-) represents a regular file. The remaining nine characters are grouped into three sets of three, representing the access rights for the **Owner**, the **Group**, and **Others**.

In this specific example, the owner has `rw-` (Read and Write) access, while both the group and others are restricted to `r--` (Read only). This configuration ensures that while the content is visible to everyone on the system, only the specific owner has the authority to modify or delete it, providing a fundamental layer of system security.

B. chmod

The `chmod` command is used to change file permissions. It allows you to control who can read, write, or execute a file. Permissions can be set using symbolic mode or numeric (octal) mode.

Numeric mode is based on binary values:

- read (`r`) = 4
- write (`w`) = 2
- execute (`x`) = 1

These values are added together for each permission group.



```
praktikumso@localhost:~/Documents$ chmod 755 Latihan1.txt
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 praktikumso praktikumso 0 Jan 17 11:28 Latihan1.txt
```

The command `chmod 755` utilizes a numeric (octal) system to define specific access rights for three different user categories. The narrative explanation for each digit is as follows:

- **The First Digit (7):** Represents the permissions for the **Owner** (the user who owns the file). The value 7 is calculated by adding 4 (read) + 2 (write) + 1 (execute), giving the owner full control over the file.
- **The Second Digit (5):** Represents the permissions for the **Group**. The value 5 is the sum of 4 (read) + 1 (execute), which means group members can view and run the file but are restricted from modifying its content.
- **The Third Digit (5):** Represents the permissions for **Others** (everyone else on the system). Like the group, they are granted only **read and execute** access based on the same 4 + 1 calculation.

In summary, the **755** configuration ensures system security by allowing only the owner to make changes, while still permitting other users to access and use the file as needed.

C. chown

The `chown` command changes the owner of a file or directory. This command is typically restricted to root or users with administrative privileges.

The most common way to check the owner and group of a file is by using the `ls -l` command. When you run this command, the output displays several columns of information:

- **The Third Column:** This lists the **Owner** (the specific user account that owns the file).
- **The Fourth Column:** This lists the **Group** (the group associated with the file).

```
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 praktikumso praktikumso 0 Jan 17 11:28 Latihan1.txt
```

This output confirms that the user `praktikumso` is the owner and it belongs to the `praktikumso` (its primary group) group. To change only the user who owns a file, use the following syntax: `sudo chown username filename`.

```
praktikumso@localhost:~/Documents$ sudo chown nayla Latihan1.txt
[sudo] password for praktikumso:
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 nayla praktikumso 0 Jan 17 11:28 Latihan1.txt
```

Based on the output shown above, *Latihan1.txt* is owned by the user “nayla”.



You can update both the owner and the group association in a single command by using a colon (:) to separate the names: `sudo chown username:groupname filename`.

```
praktikumso@localhost:~/Documents$ sudo chown nayla:praktikan Latihan1.txt
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 nayla praktikan 0 Jan 17 11:28 Latihan1.txt
```

This is particularly useful when moving a file to a different department or collaborative workspace.

In many cases, you will need to change the ownership of a directory and all the files inside it at once. Instead of changing each file individually, you can use the **Recursive flag (-R)**. This ensures that every sub-directory and file within the target folder is updated to the new owner and group simultaneously. The standard format for changing ownership across a directory tree is to use the **-R** (Recursive) flag. This ensures that the new owner and group are applied from the top-level folder down to every single sub-file and sub-directory. General syntax for applying this is `sudo chown -R [user]:[group] [directory_path]`.

```
praktikumso@localhost:~$ sudo chown -R raven:developers /home/praktikumso/
dev_project
praktikumso@localhost:~$ ls -lR /home/praktikumso/dev_project
/home/praktikumso/dev_project:
total 0
-rw-r--r--. 1 raven developers 0 Feb 17 23:44 app.py
```

As shown in the image above, the **Recursive (-R)** flag ensures that ownership changes are not limited to the parent folder. Notice that after running `chown -R`, the file `app.py`—which is located inside the directory—automatically updates to the new owner (**raven**) and group (**developers**) without needing a separate command.

D. chgrp

The `chgrp` (Change Group) command is specifically used to modify the group ownership of a file or directory. This tool is essential when you need to manage shared files among users who belong to the same group without changing the individual owner of the file.

For example, if the user "praktikumso" wants to share a script with everyone in the "praktikan" group, they would run `sudo chgrp praktikan Latihan2.txt`.




```
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 nayla      praktikan  0 Jan 17 11:28 Latihan1.txt
-rw-r--r--. 1 praktikumso praktikumso 0 Jan 17 11:59 Latihan2.txt
praktikumso@localhost:~/Documents$ sudo chgrp praktikan Latihan2.txt
praktikumso@localhost:~/Documents$ ls -l
total 0
-rwxr-xr-x. 1 nayla      praktikan 0 Jan 17 11:28 Latihan1.txt
-rw-r--r--. 1 praktikumso praktikan 0 Jan 17 11:59 Latihan2.txt
```

As shown in the output above, the group ownership of **Latihan2.txt** has been successfully changed from **praktikumso** to **praktikan**.

D. Special Permissions : The Sticky Bit

In a multi-user environment where a directory is shared among members of a group, a common security risk is that one user might delete or rename files belonging to another member. To prevent this "collaboration barrier," Linux provides a special permission called the **Sticky Bit**. When this bit is active on a directory, it ensures that even if everyone has write access to the folder, only the **file owner**, the **directory owner**, or the **root user** has the authority to delete or rename files within it. A classic example is the system's **/tmp** folder, which allows all users to create files but protects them from being deleted by others.

To apply the Sticky Bit to a shared project folder using symbolic mode, use the following command:

```
praktikumso@localhost:~$ chmod +t /home/praktikumso/dev_project
```

You can verify the Sticky Bit by checking the directory's permissions with **ls -ld**. It is indicated by a letter **'t'** (or **'T'**) at the end of the permission string:

```
praktikumso@localhost:~/dev_project$ ls -ld
drwxr-xr-t. 2 praktikumso praktikumso 6 Feb 17 23:02 .
```

3.3 File Linking (Hard Link vs Soft Link)

In the Linux filesystem, a file is essentially composed of two parts: the data stored on the disk and an **Inode**, which acts as a unique index number containing metadata about the file (such as permissions, size, and location). Linux allows you to create multiple references to the same data through a process called **Linking**. This is highly useful for system administrators who need to provide access to the same configuration file from different locations without duplicating the data.

A. Hard Link



A **Hard Link** creates a new directory entry that points to the exact same Inode as the original file. Think of it as giving one file two different names; both names are equally "real" .

- **Independence:** If you delete the original file, the data remains accessible through the hard link .
- **Limitations:** Hard links cannot be created for directories and cannot span across different partitions or filesystems.
- **Command:** `ln [target_file] [link_name]`

B. Soft Link

A **Soft Link**, or "symlink," is more like a shortcut in Windows. It is a special file that contains the path to the original file rather than pointing to the data itself .

- **Dependence:** If the original file is moved or deleted, the soft link becomes "broken" because the path it points to no longer exists.
- **Flexibility:** Unlike hard links, soft links can point to directories and can cross different filesystems.
- **Command:** `ln -s [target_file] [link_name]`

Feature	Hard Link	Soft (Symbolic) Link
Points to	The Inode (Data)	The Path (Filename)
Inode Number	Same as the original	Different from the original
If Original is Deleted	Data is still accessible	Link becomes broken
Works on Folders?	No	Yes
ls -l visual	No arrow (hard to distinguish)	Shows arrow (link -> original)

3.4 File Ownership and its Relation to Users & Groups

In Linux, every file and directory is owned by both a **user** and a **group**. This ownership is automatically assigned when a file is created and becomes the basis for how access to that file is controlled. Ownership information can be viewed using commands such as `ls -l`, where the owner and group appear alongside the permission bits.

There are two main ownership components:

- **User ownership**, which identifies the account that owns the file
- **Group ownership**, which represents a collection of users who may share access to the file

When a user attempts to access a file, Linux checks permissions in a fixed order: first as the file owner, then as a member of the file's group, and finally as others. This evaluation process ensures that the most specific permissions are applied before broader access rules.



File permissions define what actions are allowed on a file. These permissions control:

- Who is allowed to **read** the file
- Who can **modify** the file
- Who can **execute** the file as a program or script

The combination of ownership and permissions plays a critical role in **system security**. Properly configured permissions prevent unauthorized users from accessing sensitive files, protect system configuration files from accidental modification, and limit the potential impact of security breaches. By assigning the correct owner, group, and permission values, administrators can enforce access boundaries and maintain a secure and well-organized system.

EXERCISE 4 : The Gatekeeper (File Permissions)

1. **Step 1:** Create a new file named `project_plan.txt`.
2. **Step 2:** Change the **ownership** of the file so it belongs to the user `developer1` and the group `dev_team`.
3. **Step 3:** Use the **chmod** command to set permissions so that the Owner can Read/Write, the Group can Read-only, and Others have no access.
4. **Step 4:** Verify the changes by listing the file details to see the new permission strings.
5. **Step 5:** Create a **Soft Link** named `plan_shortcut` that points to `project_plan.txt`.
6. **Step 6:** Create a **Hard Link** named `plan_backup` that points to `project_plan.txt`.
7. **Step 7:** Use `ls -li` to verify and explain why `plan_backup` has the same Inode number as the original, while `plan_shortcut` does not.

EXERCISE 5: The Guardian

1. Show your current `username` and `groups`.
2. Create a file named `confidential.txt`.
3. Change the file's group to `wheel` or `bin`.
4. Set permissions to `640` (Owner: RW, Group: R, Others: None).
5. Add execute (`x`) permission for the **Owner** only.
6. Use `ls -l` to confirm the final permission string is `-rwxr-----`.

Advanced Security: Collaborative Protection

7. **Step 7:** Create a shared directory named `/home/praktikumso/collab_space` and set its permissions to `777` using `chmod`.
8. **Step 8:** Apply the **Sticky Bit** to this directory using the command `chmod +t /home/praktikumso/collab_space`.



9. **Step 9:** Use `ls -ld /home/praktikumso/collab_space` to verify that the permission string ends with a 't', indicating the Sticky Bit is active .
10. **Step 10:** Switch to a different user, try to delete a file inside that directory that you do not own, and observe the "Operation not permitted" error



PRACTICAL APPLICATIONS

PRACTICE 1

- Create a new group named **it_staff**.
- Create a new user named **staff_alpha**.
- Add that user to the **it_staff** group as their secondary group.
- Set a password for the user so they can log in to the system.
- **Set a security policy** for **staff_alpha** so their password must be changed every 30 days with a 3-day warning period using the **chage** command.
- Verify your work by viewing the contents of the **/etc/group** file and the detailed aging report using **chage -l**.

PRACTICE 2

- Create a new directory named **/home/it_vault**.
- Create a file inside it named **credentials.txt**.
- **Apply the Sticky Bit** to the **/home/it_vault** directory to ensure that only file owners can delete their own files within this shared space.
- Change the ownership of the **/home/it_vault** directory and **all of its contents recursively** so that they are owned by the user **staff_alpha** and the group **it_staff**.
- Set the access permissions using **chmod recursively** with the following requirements:
 - **Owner (staff_alpha)**: can read, write, and execute (Full Access).
 - **Group (it_staff)**: can read only (Read-only).
 - **Others**: no permissions at all (No Access).
- Display the detailed permissions of the directory and the file using the **ls -la** command to confirm the **Sticky Bit (t)** and the recursive permissions.

DEMO - CASE STUDY

The Situation

A high-stakes development team has reported that their workspace on the Linux server is compromised and inefficient. A new Lead Developer named **zamzam** has joined to stabilize the project. The project is located at **/home/praktikumso/dev_project/**. Currently, the environment is insecure, and the team is accidentally deleting each other's work.

The Problems to Identify

As the Senior Admin, you have 15 minutes to resolve the following "red flags" identified in the audit:



1. **The Ghost Lead:** The new lead, **zamzam**, has no account. Furthermore, zamzam needs **administrative (sudo) powers** to install packages, but must not be given the root password.
2. **Insecure Identity Policy:** The company just failed a security audit. New accounts must now have a **30-day password rotation policy** with a **5-day warning** to ensure credentials stay fresh.
3. **The "Accidental Deletion" Crisis:** The team reports that members of the **developers** group are accidentally deleting or renaming each other's source code files within the shared folder.
4. **The Broken Shortcut:** A critical configuration file exists in a hidden backup location, but the "shortcut" (link) in the project folder is **broken**, causing the application to crash.
5. **Ownership & Execution Chaos:** The project folder and its contents (including a script named **deploy.sh**) are owned by a former employee. The team cannot edit the files, and the script refuses to run.
6. **Privacy Leak:** Currently, any random user on the server can "peek" into the **dev_project** folder. This must be blocked immediately. The policy requires the **developers** group to have **Read and Write** access to collaborate, while **others** must have **no access** at all.
7. **Execution Failure:** The **deploy.sh** script is meant for the **Owner only** to have **Full Access (Read, Write, Execute)**, while the **Group** should only be able to **Read and Execute**.

Your Mission

1. **Identity & Privilege:** The user **zamzam** exists and returns **root** when running the command **sudo whoami**.
2. **Security Policy:** A system audit using **sudo chage -l zamzam** confirms a **Maximum Password Age of 30** and a **Warning Period of 5 days**.
3. **Ownership Integrity:** Both the **dev_project** folder and the **deploy.sh** script inside are owned by **user: zamzam** and **group: developers**.
4. **Collaborative Security:**
 - **ls -ld** shows the **'t'** attribute (Sticky Bit) is active.
 - The permission string for the folder is exactly **drwxrwx---** (no access for others).
5. **Automation & Linking:**
 - **Script Precision:** The **deploy.sh** script must have specific permissions: **Owner (rwx)** and **Group (r-x)**.
 - The **config_link** is a functional symbolic link (not broken) that points to the master configuration file.



Demo Rules

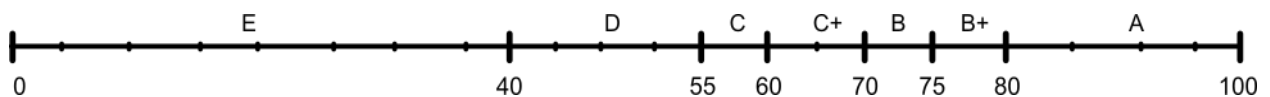
1. **No Module or Notes Access** Students must not open the module PDF, personal notes, or online documentation during the demo session.
2. **Demonstrate Core Linux Navigation Skills** Students must clearly show proficiency in:
 - User & Group Management
 - Ownership Control
 - Permission Manipulation
 - System Verification
3. **Final Verification** After completing the assigned tasks, students must verify the results (e.g., confirming a file has been moved or a directory has been deleted) to ensure the system state matches the mission requirements.
4. **Live Conceptual Assessment** During or after the demo, the assistant will ask several **key questions** to evaluate your **Conceptual Understanding**.
Note: Please prepare thoroughly, as this accounts for a significant portion of your grade.
5. **Follow Live Instructions** All actions must be performed interactively following the instructor's or assistant's directions.



ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
EXERCISE	Total 5%
Exercise 1	1%
Exercise 2	1%
Exercise 3	1%
Exercise 4	1%
Exercise 5	1%
PRACTICE	Total 20%
Practice 1 (Verbal assessment)	5%
Practice 2 (Verbal assessment)	5%
Practice Completion (Live verification)	10%
DEMO	Total 75%
Case Study Completion	35%
User & Group Management	15%
File Permissions & Ownership	15%
Group & Account Modification	10%

ASSESSMENT SCALE



- A** = (81 - 100) → Sepuh
- B+** = (75 - 80) → Very Good
- B** = (70 - 74) → Good
- C+** = (60 - 69) → Fairly Good
- C** = (55 - 59) → Fair
- D** = (41 - 54) → Poor
- E** = (0 - 40) → Bro really...

